Lab Manual
On
Compiler Design

Department of Computer Science and Engineering
Roland Institute of Technology, Surya Vihar, Berhampur – 761008.

Experiment List

1. Introduction to Tokenization and Lex / FLex
2. Basic Lex
3. Intermediate Lex
4. Advanced Lex usage
5. Yet-another Compiler-compiler

Experiment 1
Introduction to Tokenization and Lex / FLex

Tokenization
A token is a categorized block of text. Every language has rules to specifically identify textual patterns. These patterns may be words or symbols as appropriate for the specific language. These are called tokens and these need to be extracted and handled in a meaningful manner. This process of identifying textual patterns is called tokenization.

Lexeme
A lexeme is an abstract unit of morphological analysis in linguistics, that roughly corresponds to a set of forms taken by a single word. For example, in the English language, run, runs, ran and running are forms of the same lexeme, 'run'.

Lexical Analysis
Every language has its lexical rules. A lexical analyzer (lexer) processes lexemes to categorize them according to function, giving them meaning.

Lex / FLex
Lex is used to create a text file scanner and can be used for different purposes. Lex is widely used for lexical analysis. FLex or Fast Lexical analyzer is a free software destributed by GNU. On Linux based systems generally flex is supplied with a soft link from lex.

Lex Specification File
While using Lex we have to create a specification file which would specify the tokenization rules. This when processed by lex creates a language file lex.yy.c and this file can be compiled with gcc with -lfl flag. This will give rise to a binary which does tokenization.
There are the following sections in a Lex specification file

```
%{
      1) declaration section-----------
%}
      2) definitions section-----------
%%
      3) rules section  ---------------
%%
      4) user code---------------------
         -----------------------------
```

Method

The lex specification file thus created is passed through lex to create the lexer. To pass it through lex type:-

```
manik@manik:~/cdlab>lex filename.lex
Then compile the lex.yy.c file by
manik@manik:~/cdlab>gcc lex.yy.c -lfl
run it by ./a.out and press enter to insert the text to be processed manually or pass a
file containing text by the following method:-
manik@manik:~/cdlab>./a.out <filename
```

Four methods of Invocation

There are four methods of invocation of Lex. Let's explore with the simplest lex specification file.

Example 1

Simplest Lex Specification file, no patterns and no actions. So all strings match the default action, ie. printing on screen. After you are done press ^d to exit.

```
%{
%}

%%
%%
```

Example 2

Simple lex specification file with user code and the yylex funtion; but without any pattern and action specified. yylex is automatically inserted if not provided. yylex function starts the lexical analysis.

```
%{
%}

%%
%%
main(){
yylex();
return 0;
}
```

Example 3

Providing ending character instead of ^d. After entering the text type the provided end mark to end your file and start with analysis.

```
manik@manik:~/cdlab>./a.out <<quit
> abcd
> efgh
> quit
abcd
efgh
```

Example 4

Passing a file through Unix redirection

```
manik@manik:~/cdlab>./a.out <text.txt
This is a very important file.
Don't leave it open.
If somebody sees it,
all our business will be toast!
Ok?
```

Example 5

Taking a file from command line using yyin file pointer.

Files existing on the system can be taken as command line arguments like `manik@manik:~/cdlab>./a.out filename.ext` The following example takes a file from the command line. As we have not used any rules it just echoes the file on screen.

```
%{
%}

%%
%%
main(int argc, char* argv[]){
extern FILE *yyin;
yyin=fopen(argv[1],"r");
yylex();
return 0;
}
```

Experiment 2
Basic Lex

Lex can be used as a text processing tool. It can effectively be used for adding conditional formatting to the input text. However there is no special mechanism to store the processed output text. The standard Unix redirection methods can be used to store the output.

Example 5
Word substitution : Write a lex specification file to change all instances of monday to friday from a string.

Program

```
%{
%}
%%
monday  {printf("friday");}
%%
main(){
yylex();
return 0;
}
```

Output

```
manik@manik:~/cdlab> ./a.out
sunday is good
sunday is good
monday is good
friday is good
```

Problem 1
Write a lex specification file to replace all occurrences of printf and scanf with output and input in the input C source file using command line arguments.

Example 6
Write a lex specification file to change all characters in the input to stars '*'.

Program

```
%{
%}
%%
.  {printf("*");}
%%
main(){
yylex();
return 0;
}
```

Output

```
manik@manik:~/cdlab> ./a.out<<end
> my secret msg goes here
> star star star
> ok
> end
**********************
**************
**
```

Problem 2
Write a lex specification file to change all characters in the input to stars except the characters 'a' and 'b'.

Example 7
Write a lex specification file to show a file in double spacing.

Program

```
%{
%}
%%
\n  {printf("\n\n");}
%%
main(int argc, char* argv[]){
extern FILE *yyin;
yyin=fopen(argv[1],"r");
yylex();
return 0;
}
```

Output

```
manik@manik:~/cdlab> cat new.txt
this is my single
space text which
will be converted
to double space
manik@manik:~> ./a.out new.txt
this is my single

space text which

will be converted

to double space
```

Problem 3
Write a lex specification file to remove newline characters from a file.

Defining a named pattern
Lex provides a convenient method to define named patterns. Naming a pattern is advantageous as it helps to understand a relatively complex pattern through a meaningful name. Names must be single words and on a single line only one pattern can be defined.

Example 8
Write a lex specification file to remove every line starting with 'a' from an input file.

Program

```
%{
%}
aline a.*\n
%%
{aline}  {}
%%
main(int argc, char * argv[]){
extern FILE *yyin;
yyin=fopen(argv[1],"r");
yylex();
return 0;
}
```

Output

```
manik@manik:~> cat text.txt
abcd efgh
ijkl mnop
abcd efgh
ab
jsdkjf
ok
manik@manik:~>./a.out text.txt
ijkl mnop
jsdkjf
ok
```

The matched pattern yytext

We can use yytext as the text that matches the criteria for which we are searching. This can be used to modify the text in several manners.

Declaration section can be used to declare variables which can be used throughout the program.

Example 8

Write a lex specification file to read a file and add line numbers to it.

As the line ending mark we are using .*\n (dot for any character, star for multiples of dot and \n for new line at the end of the pattern. Here `number` is an integer which will be used for counting lines and linemark is a definition for the pattern to be searched.

Program

```
%{
int number=1;
%}
linemark .*\n
%%
{linemark}  {printf("%4d#  %s",number+
+,yytext);}
%%
main(int argc, char* argv[]){
extern FILE *yyin;
yyin=fopen(argv[1],"r");
yylex();
return 0;
}
```

Output

```
manik@manik:~/cdl> cat text.txt
This is a very important file.
Don't leave it open.
If somebody sees it,
all our business will be toast!
Ok?
manik@manik:~/cdl> ./a.out <text.txt
   1#  This is a very important file.
   2#  Don't leave it open.
   3#  If somebody sees it,
   4#  all our business will be toast!
   5#  Ok?
```

Example 9

Write a lex specification file to enclose every line in paranthesis. (Use of NOT)

Program

```
%{
%}
linemark .*[^\n]
%%
{linemark}  {printf("( %s )",yytext);}
%%
main(int argc, char* argv[]){
extern FILE *yyin;
yyin=fopen(argv[1],"r");
yylex();
return 0;
}
```

Output

```
manik@manik:~/cdl> cat text.txt
This is a very important file.
Don't leave it open.
If somebody sees it,
all our business will be toast!
Ok?
manik@manik:~/cdl> ./a.out text.txt
( This is a very important file. )
( Don't leave it open. )
( If somebody sees it, )
( all our business will be toast! )
( Ok? )
```

Example 10

Write a lex specification file to enclose every word in paranthesis.

Program

```
%{
%}
wordmark [^ \n]*
%%
{wordmark}  {printf("( %s )",
             yytext);}
%%
main(int argc, char* argv[]){
extern FILE *yyin;
yyin=fopen(argv[1],"r");
yylex();
return 0;
}
```

Output

```
manik@manik:~/cdl> cat text.txt
This is a very important file.
Don't leave it open.
If somebody sees it,
all our business will be toast!
Ok?
manik@manik:~/cdl> ./a.out text.txt
( This ) ( is ) ( a ) ( very ) ( important ) ( file. )
( Don't ) ( leave ) ( it ) ( open. )
( If ) ( somebody ) ( sees ) ( it, )
( all ) ( our ) ( business ) ( will ) ( be )
( toast! )
( Ok? )
```

Problem 4

Write a lex specification file to count the number of words present in a file.

Problem 5

Write a lex specification file to print every word present in a file in a separate line with extra information about the word like the number of the word and length of the word.

Experiment 3
Intermediate Lex

Example 11
Write a lex specification file to put in paranthesis the first word of every line.

Program

```
%{
%}
word [^ \n\t]+
rest [ ]*([^\n])+\n
%%
{word}  {printf("<%s>",yytext);}
{rest} {ECHO;}
%%
main(int argc, char * argv[]){
extern FILE *yyin;
yyin=fopen(argv[1],"r");
yylex();
return 0;
}
```

Output

```
manik@manik:~/cdl> ./a.out<text.txt
<This> is a very important file.
<Don't> leave it open.
<If> somebody sees it,
<all> our business will be toast!
<Ok?>
```

Example 12
Write a lex specification file to put in paranthesis the last word of the file

Program

```
%{
%}
space [ ]*
lword ([^ \n\t])+[ ]*\n
%%
{lword} {
        yytext[strlen(yytext)-1]='\0';
        printf("( %s )\n",yytext);}
{space} {ECHO;}
%%
main(int argc, char * argv[]){
extern FILE *yyin;
yyin=fopen(argv[1],"r");
yylex();
return 0;
}
```

Output

```
manik@manik:~/cdlab> ./a.out text.txt
This is a very important ( file. )
Don't leave it ( open. )
If somebody sees ( it, )
all our business will be ( toast! )
( Ok? )
```

Example 13

Write a lex specification file to Replace spaces of 1 to 4 spaces in a file with a ~ symbol and spaces of 5 to 8 with @ symbol.

Program

```
%{
%}
space124 [ ]{1,4}
space528 [ ]{5,8}
%%
{space124} {printf("%c",'~');}
{space528} {printf("%c",'@');}
%%
main(int argc, char * argv[]){
extern FILE *yyin;
yyin=fopen(argv[1],"r");
yylex();
return 0;
}
```

Output

```
manik@manik:~/cdlab> cat text.txt
abcd    efgh       ijklmno       pqr s  t

manik@manik:~/cdlab> ./a.out text.txt
abcd~efgh@ijklmno@pqr~s~t
```

Example 13

Write a lex specification file to replace characters between a to k with * and numbers between 3 to 6 with ~ symbol.

Program

```
%{
%}
pattern1 [a-k]
pattern2 [3-6]
%%
{pattern1} {printf("%c",'*');}
{pattern2} {printf("%c",'~');}
%%
main(int argc, char * argv[]){
extern FILE *yyin;
yyin=fopen(argv[1],"r");
yylex();
return 0;
}
```

Output

```
manik@manik:~/cdlab> cat text.txt
abcd efgh ijkl mnop 1234 5678

manik@manik:~/cdlab> ./a.out text.txt
**** **** ***l mnop 12~~ ~~78
```

Example 14

Write a lex specification file to find a multiple pattern of abc in a text file and replace with a ~. For example abc abcabc abcabcabc …. will be replaced.

Program

```
%{
%}
pattern (abc)*
%%
{pattern} {printf("%c",'~');}
%%
main(int argc, char * argv[]){
extern FILE *yyin;
yyin=fopen(argv[1],"r");
yylex();
return 0;
}
```

Output

```
manik@manik:~/cdlab> cat text.txt
abcd efgh abcabcdefgh abcabcabcdefgh

manik@manik:~/cdlab> ./a.out text.txt
~d efgh ~defgh ~defgh
```

Example 15

Write a lex specification file to extract comments from a C source file

Program

```
%{
int flag=0;
%}
comment \/\*.*\*\/
begin \/\*
end \*\/
%%
{comment} {printf("%s :Single line
comment",yytext);}
{begin} {printf("%s :Comment
begin",yytext);flag=1;}
{end} {printf("%s :Comment
close",yytext);flag=0;}
. {if(flag==1) printf("%s",yytext);}
%%
main(int argc, char * argv[]){
extern FILE *yyin;
yyin=fopen(argv[1],"r");
yylex();
return 0;
}
```

Output

```
manik@manik:~/cdlab> cat abcd.c
/*simple hello world program*/
#include<stdio.h>
int main(){ /*main function*/
printf("hello world"); /* this line
actually prints to the console*/
return 0;
}

manik@manik:~/cdlab> ./a.out abcd.c
/*simple hello world program*/ :Single
line comment

/*main function*/ :Single line comment
/* :Comment begin this line
actually prints to the console*/ :Comment
close
```

Problem 6: eliminate all comments from the C source file.